Aureus Agent: Verifiable Autonomous Trading Architecture

A Cryptographically Sealed, Contract-Bounded Quantitative Execution Framework on BNB Chain

Lead Architect:	Khairan Noor Fadhlillah	**Contact Email:**	dev@khairan.tech
LinkedIn:	/in/khairannoorfadhlillah	**GitHub Repository:**	github.com/codesbykhairannoor/aureus-agent-
Live Web App:	aureus-agent.vercel.app	**License:**	MIT Open Source

1. Executive Summary & Problem Formulation

The proliferation of autonomous AI trading agents in Web3 has exacerbated a fundamental trust deficit. Traditional algorithmic bots operate as closed black boxes: track records are selectively published via screenshots or private messaging channels, winning calls are retroactively celebrated, and losing positions are deleted or obscured. Furthermore, algorithmic bots execute without programmatic safety ceilings—an unexpected API crash, oracle manipulation, or execution loop hallucination frequently leads to total capital depletion.

Aureus Agent replaces subjective trust with immutable on-chain mathematics natively on **BNB Smart Chain (BSC Mainnet)**. Capital execution is strictly subordinated to smart contracts: predictive calls are cryptographically sealed prior to market moves, portfolio equity is anchored to a non-negotiable circuit breaker contract, and every transaction is audited by a fail-closed pre-flight security solver.



2. The Three Architectural Pillars

Aureus Agent decouples computational market intelligence from financial signing authority. Even if the off-chain neural model produces a flawed signal, the on-chain smart contracts prevent catastrophic execution.

Pillar I: Cryptographic Commit-Reveal Prediction Registry

Deployed at `0xcA9499a2d20cFAa98f9Bc3b2F1386A70f51c2FEB` on BSC Mainnet, the `CommitRevealPredictionRegistry` establishes an audit trail that cannot be backfilled:

- **Commitment Phase (T_0):** Before submitting an order, the agent generates a cryptographic commitment hash:

  ```
  C = keccak256(abi.encodePacked(agentId, symbol, direction, targetPrice, expiryBlock, salt))
  ```

  The hash is submitted to BSC Mainnet via `commitPrediction(C)` and mined into a block, creating an immutable timestamp prior to market resolution.
- **Execution & Lock Phase (T_1):** The position is opened or simulated. The plaintext prediction remains hidden on-chain to eliminate frontrunning and copy-trading vulnerabilities.
- **Reveal Phase (T_2):** Once the interval closes, the agent reveals parameters and the secret salt. The contract verifies `keccak256(...) == C`. Upon match, the outcome is scored against decentralized oracle feeds and recorded on the public BSC result ledger (`0x77511fEFF4c0CA8bD5aeA8d64dC6a8dAe88C0744`).

> **Cryptographic Guarantee**
> Because *C* is committed to BSC Mainnet prior to price outcome, the agent cannot retroactively alter its trade prediction or claim calls it did not commit. A stranger can audit the entire track record on BscScan with zero trust assumptions.

Pillar II: Chain-Anchored Risk Governance (`RiskGovernor.sol`)

Automated trading agents frequently fail due to uncontrolled drawdown during market regime shifts. Aureus Agent enforces programmatic capital preservation via `RiskGovernor.sol` (deployed on BSC Mainnet at `0xFACBc0a9b5a7dd34BfD0B91e1102dF4E72b9e43f`).

The contract maintains an immutable risk ceiling where `maxDrawdownBps = 1400` (14.0%). Account high-water mark equity is continuously recorded by an automated keeper (`0x064Af3880d562720963bba400B51F95d45AF91d3`). If drawdown exceeds 14%, the contract automatically transitions to the `HALTED` state, revoking token approvals on-chain.

```
Fractional Kelly Allocation Formula: f* = min( κ · [ (p · b - q) / b ], f_drawdown_ceiling )
where: p = 0.4992 (empirical win rate), q = 0.5008, b = 1.82 (payoff ratio), κ = 0.25 (quarter-Kelly safety factor)
```

The internal sizing engine sizes each leg so that a sequence of adverse outcomes stops trading before the 14% on-chain threshold is reached, providing a double-layered defense mechanism.

Pillar III: Fail-Closed Pre-Flight Security Solver

Prior to broadcasting any DEX transaction or granting approvals, the autonomous execution loop passes the order payload through `tx_security_solver.py`. Execution aborts immediately if any check fails:

Security Gate	Evaluation Methodology	Pass Criteria & Enforcement
1. Honeypot & Tax Guard	Local EVM fork simulation of buy/sell transfers	Combined buy/sell tax ≤ 2.0%; transfer function must not revert.
2. Allowance Containment	On-chain query of ERC-20 allowances on BSC	Strictly zero unlimited approvals (`2^256-1`); exact trade amount only.
3. Dynamic Slippage Cap	PancakeSwap pool depth & price impact calculation	Max price slippage ≤ 0.50% based on simulated swap output.
4. Pre-Flight Call Trace	`eth_call` gas consumption & balance delta verification	Execution gas ≤ 350,000 wei; expected asset balance must increase.
5. MEV & Frontrunning Guard	Private validator RPC submission	Bypasses public mempool; rejects transactions when spread > 0.2%.
6. Oracle Deviation Gate	PancakeSwap spot price vs. Chainlink/Pyth price feed	DEX spread $\lvert P_{spot} - P_{oracle} \rvert$ must not exceed 0.80%.

3. Interoperability, Identity & Ecosystem Standards

ERC-8004 On-Chain Agent Identity

Aureus Agent holds minted Token ID **#131181** in the official ERC-8004 Registry on BSC (`0x8004A169FB4a3325136EB29fA0ceB6D2e539a432`). This on-chain passport binds the agent wallet to its verifiable audit record and public metadata URI.

HTTP 402 Machine-Payable Feeds

Third-party agents can query Aureus Agent's verified conviction signals over HTTP 402 using EIP-3009 signed authorizations. Micro-fees are settled autonomously in BNB or USDT without human intermediary intervention.

4. Verifiable Dataset Commitment & Empirical Honesty

Unlike conventional trading models that claim unverified 80%+ win rates, Aureus Agent's edge is fitted against a historical base of **198,248 resolved outcomes across 42 symbols** on BSC. The Merkle root of this production dataset was sealed on BSC Mainnet to prevent data tampering:

BSC Mainnet Dataset Seal Transaction:
`0xc3501e1e40954a8b4ca8da36a1018c7d32016e5aa823e63cdc02a5794a9917bc`
Sealed Merkle Root: `0x42ce0ec21ca92a98f5b3a696a417255ee1adb739f240b3649abb6b736de183d5`
Realized Win Rate: ~49.92% | **Realized Payoff Ratio:** 1.82:1 | **Expected Value:** Net positive after all swap fees & slippage

5. Verified BSC Mainnet Deployments

Every contract listed below is deployed on **BNB Smart Chain (Mainnet · Chain ID 56)** with public, verified source code accessible on BscScan:

Contract Component	Mainnet Address (BSC Chain 56)	Explorer Verification
CommitRevealPredictionRegistry	`0xcA9499a2d20cFAa98f9Bc3b2F1386A70f51c2FEB`	View Verified Code on BscScan ↗
RiskGovernor (Drawdown Breaker)	`0xFACBc0a9b5a7dd34BfD0B91e1102dF4E72b9e43f`	View Verified Code on BscScan ↗
ERC-8004 Identity Registry	`0x8004A169FB4a3325136EB29fA0ceB6D2e539a432`	View Token #131181 on BscScan ↗
Result Settlement Ledger	`0x77511fEFF4c0CA8bD5aeA8d64dC6a8dAe88C0744`	View Ledger on BscScan ↗
Agent Execution Wallet	`0xD5df700Ed5355f0c778159592a072B8773faE1CC`	View Wallet on BscScan ↗

6. BNB Chain Builder Grant Roadmap ($15,000 Allocation)

The $15,000 Builder Grant funds the stabilization and developer distribution of our working BSC architecture across three phased milestones:

Milestone 1 — Telemetry & UI Hardening ($5,000 | Month 1)

Optimizing RPC query caching for BSC Mainnet, hardening the live web cockpit (aureus-agent.vercel.app), and embedding direct 1-click BscScan links for all historical commit-reveal verification transactions.

Milestone 2 — Multi-Asset Strategy Modules ($5,000 | Month 2)

Configuring tested parameter presets for top liquid BSC pairs (BNB/USDT, BTCB/USDT, ETH/USDT) under verified RiskGovernor protection, publishing automated paper-trading logs and unit test suites on GitHub.

Milestone 3 — Open Developer Quickstart, Docker Distribution & Video Showcase ($5,000 | Month 3)

Publishing complete developer documentation, a 1-command Docker container, and a 5-minute video walkthrough demonstrating how any Web3 builder can deploy their own verifiable agent on BSC in under 15 minutes.

7. Lead Architect Profile & Official Project References

Khairan Noor Fadhlillah — Lead Web3 Systems Architect & Quantitative Engineer.
Specializing in EVM smart contract architecture, high-frequency decentralized exchange routing, and automated risk systems.

Official Email: dev@khairan.tech

LinkedIn Profile: /in/khairannoorfadhlillah

GitHub Organization: github.com/codesbykhairannoor/aureus-agent-

Live Production DApp: aureus-agent.vercel.app